3.3 Fundamentals of Data Representation

AQA GCSE Computer Science (8525) — Paper 2

Computers store **everything** — text, numbers, images, sound, programs — as patterns of binary digits (0s and 1s). This topic explains how those patterns are organised into useful information: how numbers are written in different bases, how text, pictures and sound are encoded, and how files can be made smaller without (or with) losing detail. Understanding these ideas is essential because every later topic (networks, CPUs, storage) manipulates the same raw binary data.

3.3.1 Number Bases

> A **number base** (or **radix**) is the number of distinct digits used by a number system, including 0. The place value of each digit is a power of the base. AQA requires you to know three bases: **denary** (base 10, the everyday number system), **binary** (base 2, used inside every digital computer), and **hexadecimal** (base 16, used as a shorter, human-readable form of binary).

Denary (Base 10)

Denary (also called **decimal**) is the system humans use every day. It has 10 digits: `0, 1, 2, 3, 4, 5, 6, 7, 8, 9`. The place values are powers of 10: units, tens, hundreds, thousands, and so on. The number `3 472` means `3×1000 + 4×100 + 7×10 + 2×1`.

Binary (Base 2)

Binary has only **two digits**: `0` and `1`. Each digit is called a **bit** (short for *binary digit*). Computers use binary because every component inside the processor and memory can be in one of two physical states — for example, a capacitor charged or empty, a voltage high or low, a switch on or off. Two-state systems are cheap, reliable and easy to build; a system with 10 distinct voltage levels for denary would be far more error-prone.

Place values in binary are powers of 2. Reading an 8-bit binary number from left to right, the place values are:

Bit position	7	6	5	4	3	2	1	0
Place value	128	64	32	16	8	4	2	1
Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

8-bit Binary Place-Value Columns

Each column doubles as you move left. Maximum value of 8 bits = 128+64+32+16+8+4+2+1 = 255.

128	64	32	16	8	4	2	1
1	0	1	1	0	1	0	1

128 + 32 + 16 + 4 + 1 = **181** → the binary pattern **10110101** equals denary **181**

Hexadecimal (Base 16)

Hexadecimal (usually shortened to **hex**) has 16 digits. The digits 0–9 have their usual values; the letters A, B, C, D, E, F represent the extra digits for 10, 11, 12, 13, 14, 15.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D
Denary	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101

Why is Hexadecimal Used?

Hexadecimal is not used because computers work in base 16 — they do not. Hex is used by *humans* when they need to read or write binary values. The reasons are:

- **Shorter to write:** a long 8-bit binary number like `11110011` becomes just `F3` in hex — four times fewer characters.
- **Easier to read and less error-prone:** a programmer is far more likely to mis-type or mis-read `10111010 11011110` than `BADE`.
- **Direct mapping to binary:** because 16 = 2^4, exactly 4 bits (a *nibble*) correspond to one hex digit, so you can convert between binary and hex without any arithmetic — just look up each nibble.
- **Compactness:** two hex digits fit in one byte, which matches how memory is organised.

Hex appears anywhere binary data must be shown to a human: memory addresses (`0x7FFE0A14`), HTML colour codes (`#1E40AF`), MAC addresses (`00:1B:44:11:3A:B7`), error codes, and the contents of a file viewed in a "hex editor".

> **Exam Tip:** When an exam asks "state one reason hexadecimal is used instead of binary", aim for a *specific* reason plus a *consequence*: "Hex is shorter — 4 binary digits become 1 hex digit — so programmers can read and write long binary values more quickly and with fewer mistakes." Avoid vague answers such as "it is easier".

3.3.2 Converting Between Number Bases

Binary → Denary

Write the 8-bit place-value columns above the binary number, then add up the place values that have a `1` beneath them.

Worked example: convert `10110110` to denary.

Place value	128	64	32	16	8	4	2	1
Binary	1	0	1	1	0	1	1	0
Contribution	128	0	32	16	0	4	2	0

Total: `128 + 32 + 16 + 4 + 2 = 182`. So `10110110`$_2$ = 182_{10}.

Denary → Binary (Repeated Halving / Subtraction)

Two reliable methods work at GCSE. The **subtraction method** is usually quicker by hand.

Subtraction method — convert 181 to 8-bit binary. Go from the largest place value (128) downwards. At each column, ask: does the remaining number fit? If yes, write `1` and subtract; if no, write `0`.

Place value	Fits in remainder?	Bit	New remainder
128	181 ≥ 128 — yes	**1**	181 − 128 = 53
64	53 ≥ 64 — no	**0**	53
32	53 ≥ 32 — yes	**1**	53 − 32 = 21
16	21 ≥ 16 — yes	**1**	21 − 16 = 5
8	5 ≥ 8 — no	**0**	5
4	5 ≥ 4 — yes	**1**	5 − 4 = 1
2	1 ≥ 2 — no	**0**	1
1	1 ≥ 1 — yes	**1**	0

Reading top to bottom: `181 = 10110101`$_2$. Always check your answer by converting back.

Repeated division method — convert 45 to binary. Divide by 2 repeatedly, writing the remainder each time. Read the remainders *upwards*.

Step	Division	Quotient	Remainder
1	45 ÷ 2	22	1
2	22 ÷ 2	11	0
3	11 ÷ 2	5	1
4	5 ÷ 2	2	1
5	2 ÷ 2	1	0
6	1 ÷ 2	0	1

© RAG Learning · Not for redistribution

Read remainders bottom-to-top: `101101`. Pad to 8 bits with leading zeros: **`00101101`**$_2$ = 45.

Range of 8-bit Binary

An 8-bit number has 2^8 = 256 possible patterns, from `00000000` (0) to `11111111` (255). The **range is 0 to 255 inclusive**. In an AQA exam, if you are asked to convert a denary number greater than 255 to 8 bits, it cannot be done — the value has *overflowed*.

Denary ↔ Hexadecimal

Denary → hex. For a value in the range 0–255 it is easiest to go via binary: split the 8 bits into two 4-bit nibbles and look up each nibble in the hex table. Alternatively, divide by 16: the quotient is the first hex digit, the remainder is the second.

Worked example: convert 181 to hex.

1. We already showed 181 = `10110101`$_2$.
2. Split into nibbles: `1011 | 0101`.
3. Look up each nibble: `1011` = B; `0101` = 5.
4. Result: `181 = B5`$_{16}$.

Division method: 181 ÷ 16 = 11 remainder 5. 11 is B, the remainder 5 is 5, so the result is `B5`.

Hex → denary. Multiply each hex digit by its place value (powers of 16) and add.

Worked example: convert `2F` to denary.

Place value	16	1
Hex digit	2	F (= 15)
Contribution	2 × 16 = 32	15 × 1 = 15

Total = 32 + 15 = **47**. So `2F`$_{16}$ = 47_{10}.

Binary ↔ Hexadecimal

This is the **easiest** conversion because 1 hex digit = 4 binary bits. No arithmetic is needed — just group and look up.

Binary → hex. Group the bits into nibbles of 4, starting from the *right*. Pad the leftmost group with leading zeros if needed. Convert each nibble to its hex digit.

Worked example: convert `11011110`$_2$ to hex.

Nibble	1101	1110
Denary	13	14
Hex	D	E

Result: `11011110`$_2$ = **`DE`**$_{16}$. (This value, 222 in denary, is how the word "DEAD" and "BEEF" in hex got famous as programmer memory markers.)

Hex → binary. Replace each hex digit with its 4-bit binary nibble.

Worked example: convert `A7` $_{16}$ to 8-bit binary.

Hex	A	7
Binary nibble	1010	0111

Result: `A7` $_{16}$ = **`10100111`** $_{2}$.

> **Exam Tip:** Always **pad** binary answers to the full number of bits requested (normally 8). Writing `101101` when an 8-bit answer is asked for loses marks; the correct answer is `00101101`. Similarly, always group binary into nibbles *from the right* when converting to hex, not from the left.

3.3.3 Units of Information

> A **bit** is a single binary digit — 0 or 1 — and is the smallest possible unit of data in a computer. Larger amounts of data are measured in bytes and multiples of bytes. The AQA specification uses the **decimal (SI) prefixes**: 1 kB = 1000 B (not 1024 B). Some textbooks still use the older binary definition (1 KB = 1024 B), so read the question carefully.

Hierarchy of Units

Unit	Symbol	Size	Notes
Bit	b	1 binary digit (0 or 1)	The smallest unit. Represents one of two possible values.
Nibble	—	4 bits	Half a byte. Represents 16 values. Maps directly to 1 hexadecimal digit.
Byte	B	8 bits	Represents 256 values. Historically enough to hold one character of text (ASCII).
Kilobyte	kB	1 000 B = 10^3 B	A short text document. (AQA uses 1 kB = 1000 B.)
Megabyte	MB	1 000 kB = 10^6 B	A high-quality photo, several minutes of MP3 audio.
Gigabyte	GB	1 000 MB = 10^9 B	A DVD film, a large game install.
Terabyte	TB	1 000 GB = 10^{12} B	A typical external hard drive.

> **Exam Tip:** To go from a smaller unit to a larger one, **divide by 1000**; to go from larger to smaller, **multiply by 1000**. For example: 4 500 000 B ÷ 1000 = 4500 kB ÷ 1000 = 4.5 MB. Do not mix bits and bytes — 8 bits = 1 byte, so "500 Mb" (megabits) is only 62.5 MB (megabytes). Network speeds are usually quoted in bits per second; file sizes in bytes.

Estimating File Sizes

Working out how many bytes, kilobytes or megabytes a file occupies is a common exam question. The general method is always:

1. Work out the total number of **bits** needed, using the appropriate formula (character code, image or sound).
2. Divide by 8 to convert bits to bytes.
3. Divide by 1000 once to get kB, twice to get MB, three times to get GB.

Quick example: a plain text message of 400 ASCII characters. Each character is 7 bits in standard ASCII (often padded to 8 bits = 1 byte). Size = 400 × 1 byte = **400 B = 0.4 kB**.

3.3.4 Binary Arithmetic

AQA requires you to be able to **add two 8-bit binary numbers**, handling carries correctly, and to recognise when an addition causes **overflow**.

Rules of Binary Addition

A	B	Sum bit	Carry out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	**1** (carry to the next column)
Plus, with a carry in: 1 + 1 + 1 = **1 sum, 1 carry**.			

So the only rule to remember is that **1 + 1 = 10** in binary (like 9 + 1 = 10 in denary). Any time two `1`s combine in a column, you write `0` and carry `1` to the next column on the left.

Worked Example 1 — No Overflow

Add `01011010` (90) and `00110101` (53).

```
Carries:    0 1 1 0 0 0 0 0
            -----------------
  01011010  (90)
+ 00110101  (53)
  -----------------
  10001111  (143)   ← expected: 90 + 53 = 143 ✓
```

Working column-by-column from the right:

- Column 0: 0 + 1 = 1, no carry.
- Column 1: 1 + 0 = 1, no carry.
- Column 2: 0 + 1 = 1, no carry.
- Column 3: 1 + 0 = 1, no carry.

- Column 4: 1 + 1 = 10 → write 0, carry 1.
- Column 5: 0 + 1 + carry 1 = 10 → write 0, carry 1.
- Column 6: 1 + 0 + carry 1 = 10 → write 0, carry 1.
- Column 7: 0 + 0 + carry 1 = 1, no carry out of the byte.

Result: `10001111`$_2$ = 143_{10}. ✓

Worked Example 2 — Overflow

> **Overflow** happens when the result of an arithmetic operation is too large to fit in the number of bits available. For 8-bit unsigned addition, overflow occurs whenever the correct answer is greater than 255, i.e. a final carry-out is generated from the leftmost column. The hardware will either *truncate* the extra bit (giving a wrong answer) or raise an overflow *flag* that the program can check.

Add `11010110` (214) and `01011001` (89). The correct denary total is 303, which is greater than 255 and so does not fit in 8 bits.

```
 Carries:  1 1 1 1 0 1 1 0
           -----------------
(carry out) 1
  11010110   (214)
+ 01011001   ( 89)
  -----------------
  00101111   ( 47)   ✗ wrong! 214 + 89 = 303, which needs 9 bits.
```

The 9th bit is the **carry-out** of the byte. Because only 8 bits are stored, the final pattern is `00101111` (47) — this is 303 − 256 = 47. An overflow flag would be set to signal that the arithmetic was incorrect.

> **Exam Tip:** A question asking "has overflow occurred?" can be answered by checking two things: (1) is there a final carry-out beyond the leftmost bit? If yes, overflow has occurred. (2) Does the denary answer exceed 255 (for unsigned 8-bit)? If yes, overflow has occurred. State both checks if space allows.

3.3.5 Binary Shifts

> A **binary shift** moves every bit in a binary number a fixed number of positions to the left or to the right. AQA requires you to understand **logical** shifts only: a logical left shift inserts a 0 on the right and discards the leftmost bit; a logical right shift inserts a 0 on the left and discards the rightmost bit.

Logical Left Shift (Multiplication by 2)

Each left shift by one position **multiplies the number by 2**. Shifting left by *n* places multiplies by 2^n. This is because each bit that moves left is now in a column worth twice as much.

Worked example: shift `00010110` (22) left by 2 places.

Step	Binary	Denary

Before	`00010110`	22
After 1 left shift	`00101100`	44 (= 22 × 2)
After 2 left shifts	`01011000`	88 (= 22 × 4)

Logical Right Shift (Integer Division by 2)

Each right shift by one position **divides the number by 2**, discarding any remainder (integer division). Shifting right by *n* places divides by 2^n.

Worked example: shift `01100100` (100) right by 3 places.

Step	Binary	Denary
Before	`01100100`	100
After 1 right shift	`00110010`	50
After 2 right shifts	`00011001`	25
After 3 right shifts	`00001100`	12 (not 12.5 — remainder lost)

Risks of Binary Shifts — Lost Bits

Because shifts push bits off the end of the byte, **information can be lost**:

- **Left shift:** if a `1` bit is pushed off the left end, the value has overflowed and the result is wrong. Example: `10000001` (129) shifted left by 1 becomes `00000010` (2), not 258. The top bit was lost.
- **Right shift:** any `1`s pushed off the right are discarded, so the shift acts as *integer* division, not exact division. Example: `00000111` (7) shifted right by 1 becomes `00000011` (3), not 3.5.

> **Exam Tip:** When asked "what is the effect of a logical left/right shift by n?" give *both* the mathematical effect ("multiplies / divides by 2^n") and the risk ("bits shifted off the end are lost, which can cause overflow or loss of precision"). Two clear points typically score both marks.

3.3.6 Character Encoding

> A **character set** is the agreed list of characters that a system can represent, each paired with a unique **character code** (or **code point**) — the binary number used to store that character. Without an agreed character set, text typed on one computer could not be read correctly on another.

ASCII (American Standard Code for Information Interchange)

Standard ASCII uses **7 bits** per character, giving 2^7 = 128 possible codes (0 to 127). These codes cover:

- The uppercase letters A–Z (codes 65–90)
- The lowercase letters a–z (codes 97–122)

- The digit characters 0–9 (codes 48–57)
- Common punctuation and symbols (space, . , ! ? @ # etc.)
- Non-printing *control* characters for early teletypes (codes 0–31 and 127) — e.g. carriage return (13), line feed (10), tab (9).

In practice each character is stored in one full **byte** (8 bits), with the top bit set to 0. This is sometimes called "7-bit ASCII stored in 8 bits".

ASCII Table Extract

Char	Denary	Binary (8-bit)	Hex
(space)	32	00100000	20
'0'	48	00110000	30
'1'	49	00110001	31
'9'	57	00111001	39
'A'	65	01000001	41
'B'	66	01000010	42
'M'	77	01001101	4D
'Z'	90	01011010	5A
'a'	97	01100001	61
'b'	98	01100010	62
'm'	109	01101101	6D
'z'	122	01111010	7A

Relationships Between Character Codes

The codes are deliberately arranged so that common operations are easy:

- **Letters run in order.** 'A' = 65, 'B' = 66, 'C' = 67 … 'Z' = 90. So to jump *n* letters along the alphabet, add *n* to the code. This makes alphabetical sorting straightforward — just compare character codes.
- **Lowercase sits 32 higher than uppercase.** 'a' = 97 = 65 + 32. Adding 32 converts an uppercase letter's code to its lowercase equivalent; subtracting 32 goes the other way.
- **Digit codes are not the same as digit values.** '7' has code 55, not 7. To turn a digit *character* into its numeric value, subtract the code of '0' (48): `'7' - '0' = 55 - 48 = 7`.

Extended ASCII (8-bit)

Using all 8 bits of a byte doubles the range to 256 codes. The extra 128 codes (128–255) were used for accented letters (é, ü, ñ), basic box-drawing graphics and currency symbols. Different manufacturers chose different encodings, which caused compatibility problems; the ISO-8859 family standardised several variants for different European languages.

Unicode

> **Unicode** is a character set that aims to represent *every* character in *every* writing system, plus mathematical symbols, emoji and historic scripts. It uses up to **16 bits (and beyond)** per code point, giving at least 65 536 possible characters; the full Unicode standard defines over a million code points. Common storage formats include UTF-8 (variable-width, ASCII-compatible) and UTF-16.

Unicode was created because ASCII (even extended) could not cover languages like Chinese, Arabic, Hindi or Japanese, which use thousands of characters. In Unicode, the first 128 code points are deliberately identical to ASCII, so existing English text is still valid.

ASCII vs Unicode

Feature	ASCII (7-bit)	Unicode
Bits per character	7 (often stored in 8)	16 or more (variable in UTF-8)
Number of characters	128	> 1 000 000 code points defined
Languages covered	English only	All major world scripts + emoji + symbols
Bytes for plain English text	1 byte per character	1 byte per ASCII character in UTF-8
Bytes for non-Latin scripts	Not supported	2–4 bytes per character
Typical use	Legacy text files, configuration files	Web pages, modern operating systems, databases

> **Exam Tip:** A common exam question is "state one advantage and one disadvantage of Unicode over ASCII". Advantage: supports almost every writing system in the world, so a single file can contain English, Chinese, Arabic and emoji together. Disadvantage: each character usually takes more bytes than ASCII, so plain English text files are larger.

3.3.7 Representing Images

Bitmap Images and Pixels

> A **bitmap** image is a rectangular grid of tiny coloured squares called **pixels** (short for *picture elements*). Each pixel is stored as a binary number representing its colour. When displayed at a small enough size, the grid of coloured pixels looks like a smooth image to the human eye.

Two measurements describe the quality of a bitmap image:

- **Resolution** — the *width* × *height* of the grid, e.g. 1920 × 1080. Higher resolution means more pixels and therefore more detail, but also a larger file.

- **Colour depth** (also called *bit depth*) — the number of bits used to store the colour of *each* pixel.

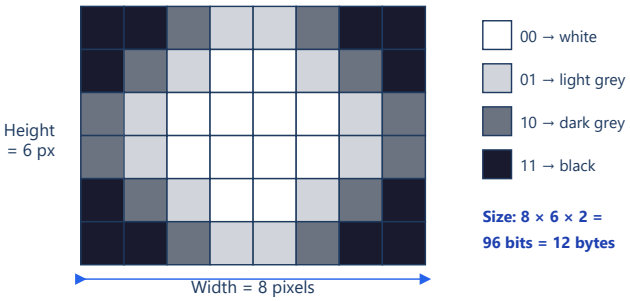
Colour Depth

With *n* bits per pixel, you can represent 2^n distinct colours.

Colour depth	Number of colours (2^n)	Typical use
1 bit	2	Monochrome (black & white) — e.g. fax images.
2 bit	4	Very simple palette, early arcade graphics.
4 bit	16	Early VGA / icon palettes.
8 bit	256	Simple web graphics (GIF), indexed-colour images.
16 bit	65 536	"High colour" — near-photographic quality.
24 bit (True Colour)	16 777 216	Standard photographic images. 8 bits each for Red, Green, Blue.

Bitmap Pixel Grid — Resolution and Colour Depth

An 8 × 6 image with 2-bit colour depth (4 shades).



File Size of a Bitmap Image

The uncompressed file size (in bits) of a pure bitmap image is:

```
File size (bits) = width × height × colour depth
```

To convert to bytes, divide by 8. To convert to kilobytes, divide by 1000 more.

Worked File Size Example

A photograph has a resolution of 800 × 600 pixels and a colour depth of 24 bits.

Step	Calculation	Result
Total pixels	800 × 600	480 000 pixels
Total bits	480 000 × 24	11 520 000 bits

Bytes	11 520 000 ÷ 8	1 440 000 B
Kilobytes	1 440 000 ÷ 1000	1440 kB
Megabytes	1440 ÷ 1000	**1.44 MB**

Metadata

> **Metadata** is "data about the data". For a bitmap image this is extra information stored alongside the pixel data that tells a program how to interpret the file. It does not include the pixel colours themselves.

Typical image metadata:

- Width and height in pixels (resolution)
- Colour depth (bits per pixel)
- File format / version
- Date and time the photo was taken
- Camera make, model and lens settings (EXIF data)
- GPS coordinates (if the camera had a GPS sensor)
- Compression method used

Without metadata the file would just be a long stream of bits and the computer would have no way of knowing, for example, whether a 48 000-bit block was a 100 × 60 image at 8 bpp or a 40 × 30 image at 40 bpp.

> **Exam Tip:** Be ready to **rearrange** the size formula. "A 1-bit bitmap file is 3000 B. If its height is 150 pixels, what is its width?" Size in bits = 3000 × 8 = 24 000. Width = 24 000 / (150 × 1) = 160 pixels. In exam answers always show the working step-by-step — method marks are available even if the arithmetic goes wrong.

3.3.8 Representing Sound

Analogue vs Digital

> A **sound wave** is a continuous, analogue vibration of air pressure. A computer cannot store an infinite continuous wave; it must convert the wave into a series of discrete numbers — a process called **sampling** or **analogue-to-digital conversion**.

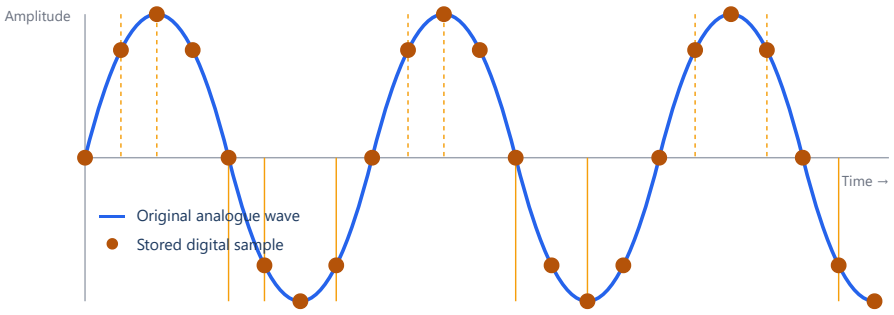
Sampling

At regular intervals, the height (amplitude) of the wave is *measured* and the result stored as a binary number. Each measurement is called a **sample**. When the samples are played back in order by a digital-to-analogue converter, they reconstruct an approximation of the original wave.

© RAG Learning · Not for redistribution

Sampling a Continuous Sound Wave

Vertical dashed lines mark each sample time. Dots are the stored values.



Sample Rate

> **Sample rate** is the number of samples taken per second of audio. It is measured in **hertz (Hz)**. Example rates: telephone speech 8000 Hz (8 kHz); CD quality 44 100 Hz (44.1 kHz); professional audio 96 000 Hz.

A *higher* sample rate captures more detail of the original wave (especially high-frequency sounds) and produces a more faithful recording, but the file is larger.

Sample Resolution (Bit Depth)

> **Sample resolution** (or **bit depth**) is the number of bits used to store each sample. CD audio uses 16 bits, giving 2^{16} = 65 536 possible amplitude levels. A higher bit depth records the amplitude more accurately, reducing "quantisation noise", but also increases file size.

File Size of a Sound File

```
File size (bits) = sample rate × bit depth × duration (s) × channels
```

Channels: 1 for mono, 2 for stereo. Divide by 8 for bytes, then by 1000 for kB.

Worked Sound File Size Example

Calculate the uncompressed file size of a 30-second stereo recording at CD quality (44 100 Hz, 16 bits per sample).

Step	Calculation	Result
Bits per second	44 100 × 16 × 2 channels	1 411 200 bits/s
Total bits	1 411 200 × 30	42 336 000 bits
Bytes	42 336 000 ÷ 8	5 292 000 B
Kilobytes	5 292 000 ÷ 1000	5292 kB
Megabytes	5292 ÷ 1000	**5.292 MB**

© RAG Learning · Not for redistribution

Effect of Increasing Sample Rate or Bit Depth

Change	Effect on quality	Effect on file size
Double the sample rate	Captures higher-frequency content more accurately — recording sounds closer to the original.	File size doubles.
Double the bit depth	Each sample's amplitude is stored to higher precision — less quantisation noise, greater dynamic range.	File size doubles (per sample).
Halve sample rate / bit depth	Lower quality, may sound muffled or "grainy".	File is half the size.

Exam Tip: If an exam question asks "why does a higher sample rate give better quality?", explain the mechanism: "Taking more samples per second means the digital recording follows the shape of the original analogue wave more closely, so fewer high-frequency details are lost when it is played back". This is worth more than simply stating "more detail".

3.3.9 Data Compression

Compression is the process of re-encoding data so that it takes fewer bits to store or transmit. It is needed because files (especially images, audio and video) are often very large, and smaller files mean less **storage space** is used and less **bandwidth** is consumed when sending them over a network.

Why Compress?

- **Storage:** more files fit on a disk, phone or cloud drive.
- **Bandwidth:** smaller files download and stream faster over slow or mobile networks.
- **Cost:** cloud providers and mobile networks charge by the gigabyte; smaller files cost less to send and host.
- **Responsiveness:** a compressed web page loads quickly, improving user experience and SEO.

There are two fundamentally different approaches: **lossy** and **lossless**.

Lossy Compression

Lossy compression permanently discards data that is considered less important — for example, high-frequency sounds a human cannot easily hear, or fine colour variations the eye struggles to distinguish. The original file cannot be reconstructed exactly; decompressing a lossy file always returns a slightly different version.

- Can achieve very large size reductions (a JPEG photo is often 10× smaller than the original).
- The quality loss is **irreversible** — save-as-JPEG-again and information is lost each time.
- Common formats: JPEG (photos), MP3 (music), MP4 / H.264 (video).

Lossless Compression

Lossless compression encodes the data so that it is smaller, but every original bit can be perfectly recovered when the file is decompressed. This is essential for text documents, executable code, spreadsheets and archives — any data where losing a bit would corrupt it.

© RAG Learning · Not for redistribution

- Typical size reduction is more modest (maybe 40–70% for text).
- Decompression produces an **identical** copy of the original.
- Common formats: PNG (images), FLAC (music), ZIP, GZIP (archives).

Lossy vs Lossless Comparison

Feature	Lossy	Lossless
Original file perfectly recoverable?	No — data is discarded	Yes — identical copy
Typical size reduction	Very large (e.g. 90%)	Smaller (e.g. 30–60%)
Quality after decompression	Lower than original	Identical to original
Used for	Photos, music, video streaming	Text, code, spreadsheets, archives
Example formats	JPEG, MP3, MP4, AAC	PNG, FLAC, ZIP, GIF
Repeat compression safe?	No — quality degrades each time	Yes — no further loss

Exam Tip: A very common exam question is "should the company use lossy or lossless, and why?" Always link your answer to the *type of data* and what the user does with it. Source code or spreadsheets must be lossless (even one wrong bit can corrupt the file). Streaming music or uploading holiday photos to social media is usually lossy, because a small drop in quality is imperceptible and saves a lot of bandwidth.

Huffman Coding

Huffman coding is a lossless compression technique that gives *short* binary codes to characters that appear *frequently* and *longer* binary codes to characters that appear rarely. On average, the total number of bits needed is less than if every character used the same fixed number of bits (e.g. 8 bits in ASCII).

How Huffman Coding Works

1. Count how often each character appears in the text (its **frequency**).
2. Put each character in its own single-node tree, labelled with its frequency.
3. Repeatedly take the two trees with the *smallest* frequencies and combine them into a new tree whose root frequency is their sum. Put this new tree back in the pool.
4. Stop when only one tree remains — this is the **Huffman tree**.
5. Label every left branch `0` and every right branch `1`. The code for each character is the sequence of 0s and 1s from the root to its leaf.

Worked Huffman Example — the word "MISSISSIPPI"

The word has 11 letters. Count the frequency of each character:

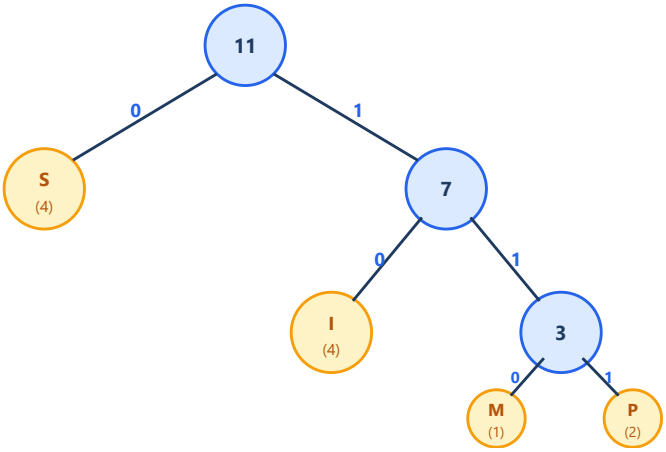
Character	I	S	P	M
Frequency	4	4	2	1

Building the Huffman tree:

1. Combine M(1) + P(2) → internal node MP(3).
2. Combine MP(3) + I(4) → internal node MPI(7).
3. Combine MPI(7) + S(4) → root(11). Done.

Huffman Tree for "MISSISSIPPI"

Left branch = 0, right branch = 1. Characters sit at the leaves.



Reading codes from root to leaf:

Character	Frequency	Huffman code	Code length (bits)
S	4	`0`	1
I	4	`10`	2
M	1	`110`	3
P	2	`111`	3

Encoded "MISSISSIPPI":

```
M I S S I S S I P P I
110 10 0 0 10 0 0 10 111 111 10   →   11010 00100 01011 11111 0
```

Total bits = 3+2+1+1+2+1+1+2+3+3+2 = **21 bits**.

Compare to fixed-length coding: with 4 different characters, 2 bits per character would be needed ($2^2 = 4$), giving 11 × 2 = **22 bits**. With 7-bit ASCII, it would be 11 × 7 = **77 bits**. Huffman beats both because the most common characters use the shortest codes.

Prefix-free property. Notice that no code is a prefix of another (`0` , `10` , `110` , `111`). This means the decoder can read the bit stream left-to-right without punctuation and know exactly when each character ends — a property called *prefix-free* or *instantaneous* decoding.

Run-Length Encoding (RLE)

> **Run-length encoding** (RLE) is a simple lossless compression technique that replaces *runs* of identical consecutive values with a single pair of numbers: the **count** and the **value**. It is highly effective when the

> data contains long repeated sequences, and useless (or worse) when the data is very varied.

Worked RLE Example — Text

Compress the string `AAAAABBBCCDAAA` using RLE.

- 5 As, 3 Bs, 2 Cs, 1 D, 3 As
- Encoded: `5A3B2C1D3A`

The original needs 14 characters; the encoded form needs only 10 — about a 29% saving. Decoding simply expands each count/value pair back out.

Worked RLE Example — Image

Consider one scanline of a black-and-white image: `WWWWWWWBBBWWWWWWWWWWWBBBBW` (26 pixels).

- 7 W, 3 B, 11 W, 4 B, 1 W
- Encoded: `(7,W)(3,B)(11,W)(4,B)(1,W)` — 10 values instead of 26.

RLE works brilliantly on images with large solid blocks of one colour (scanned forms, simple icons, faxes) but poorly on detailed photos where almost every pixel differs from its neighbour.

Comparing Huffman and RLE

Aspect	Huffman coding	Run-Length Encoding
Type	Lossless	Lossless
Idea	Frequent symbols get short codes	Runs of repeats are replaced by count + value
Best when	Some characters appear much more often than others (e.g. English text)	Data contains long sequences of the same value (e.g. simple bitmaps)
Works poorly when	All characters equally frequent	Data is varied with few repeats (can even increase size)
Requires extra data?	Yes — the code table / tree must be stored with the file	No extra table, but the count/value scheme must be agreed

> **Exam Tip:** When asked to encode a word with Huffman, always show: (1) the frequency table, (2) the tree, (3) the codes, and (4) the total bit count with a sensible comparison (e.g. vs ASCII). Accept any *valid* tree — there can be more than one correct Huffman tree for the same data, because ties may be broken in different ways. As long as the codes are prefix-free and the total bit count is minimal, full marks are usually awarded.

Summary

Topic 3.3 is really about one big idea: **every kind of information can be reduced to binary**. Numbers use place values; text uses character codes; images are grids of pixel codes; sound is a sequence of sampled amplitudes; and files can be compressed by removing redundancy (lossless) or unnecessary detail (lossy). Being fluent in the

conversions, the size formulae, and the trade-offs is the key to scoring well on Paper 2 data-representation questions.